ICT1012

OPERATING SYSTEMS

LABORATORY MANUAL

Lab1-w1: Introduction to xv6_RISC-V

AY 2025/26

BACHELOR OF ENGINEERING

IN

INFORMATION AND COMMUNICATIONS TECHNOLOGY

Contents

Part 1: Setting up xv6 OS (on RISC-V) 3
Prerequisites 3
Installing on Windows 3
Installing on macOS 3
Verifying the installation 3
Part 2: Familiarizing with xv6 OS 4
Part 3: Using gdb (GNU Debugger) 5
Starting "make qemu-gdb' from Ubuntu WSL terminal-1 5
Open another Ubuntu (WSL) terminal (terminal-2) 5
Starting 'gdb-multiarch (or riscv64-linux-gnu-gdb) -x .gdbinit' from WSL terminal-2 6
Getting familiar with "gdb" 7
Checking kernel code consistency (modifications) using "*gdb*" 9

Part 1: Setting up xv6 OS (on RISC-V)

Prerequisites

Installing on Windows

- Install Linux on local machine or use WSL 2 (Windows Subsystem for Linux 2).
  - Reference:
    Lab1_w1_Introduction_to_Linux.pdf
    WSL 2: Windows Subsystem for Linux
    Ubuntu 24.04: Ubuntu 24.04 from the Microsoft Store.
- Setup software required to run the labs
  From Windows Start menu, launch Ubuntu-24.04 app.
  Setup RISC-V versions of QEMU 7.2+, GDB 8.3+, GCC, and Binutils.

  Note:
  RISC-V: Reduced Instruction Set Computer - V
  QEMU: Quick EMUlator
  GDB: GNU Debugger
  GCC: GNU C Compiler

  In Ubuntu terminal:

```
$ sudo apt-get update && sudo apt-get upgrade

$ sudo apt-get install git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu

$ sudo apt-get install -y zip
```

Installing on macOS

- Install developer tools:

```
$ xcode-select --install
```

- Install Homebrew, a package manager for macOS:

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

  Install the RISC-V compiler toolchain:

```
$ brew tap riscv/riscv
$ brew install riscv-tools
```

- The brew formula may not link into *"/usr/local"*. You will need to update #shell's "*rc*" file (e.g. *~/.bashrc*) to add the appropriate directory to *$PATH*.

```
PATH=$PATH:/usr/local/opt/riscv-gnu-toolchain/bin
```

- Install QEMU:

```
brew install qemu
```

Verifying the installation

Check RISCV64 version:

```
$ qemu-system-riscv64 --version
QEMU emulator version 7.2.0
```

And at least one RISC-V version of GCC:

```
$ riscv64-linux-gnu-gcc --version
riscv64-linux-gnu-gcc (Debian 10.3.0-8) 10.3.0
...
```

Or

```
$ riscv64-unknown-elf-gcc --version
riscv64-unknown-elf-gcc (GCC) 10.1.0
...
```

Or

```
$ riscv64-unknown-linux-gnu-gcc --version
riscv64-unknown-elf-gcc (GCC) 10.1.0
...
```

Part 2: Familiarizing with xv6 OS

- Download '*lab.zip*' from XSiTe to a local folder "D:\ICT1012\xv6labs-w1\" using Windows File Explorer.
  Example (Windows):
  ```
  D:\ICT1012\xv6labs-w1\lab.zip
  ```
- Unzip the zip file and place the files and folders under "D:\ICT1012\ xv6labs-w1" using Windows File Explorer. After unzipping, delete "lab.zip" from "D:\ICT1012\ xv6labs-w1"
- Explore the files and folders in Ubuntu
  - Start Ubuntu terminal and mount the folder "*xv6labs-w1*" under "*ICT1012*"
  ```
  :~$
  :~$ cd /mnt/d/ICTICT1012/xv6labs-w1/
  :/mnt/d/ICT1012/xvlabs-w1$
  ```
  - Start Visual Studio Code from WSL and view the files and folders under "*xv6labs-w1*" in Visual Studio Code
  ```
  :/mnt/d/ICT1012/xvlabs-w1$ code .
  ```
- In Ubuntu terminal, run '**make qemu**' to build and run xv6
  ```
  :/mnt/d/ICT1012/xv6labs-w1$ make qemu
  qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -
smp 3 -nographic -global virtio-mmio.force-legacy=false -drive
file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-
bus.0

  xv6 kernel is booting

  hart 1 starting
  hart 2 starting
  init: starting sh
  $
  ```
  - Try '**ls**' command to list the files that '*mkfs*' includes in the initial file system, most are programs that can be run.
  ```
  $ ls
  .              1 1 1024
  ..             1 1 1024
  ```

```
README          2 2 2403
xargstest.sh    2 3 93
cat             2 4 34280
echo            2 5 33200
forktest        2 6 16192
grep            2 7 37536
init            2 8 33664
kill            2 9 33112
ln              2 10 32936
ls              2 11 36304
mkdir           2 12 33176
rm              2 13 33160
sh              2 14 54736
stressfs        2 15 34056
usertests       2 16 179360
grind           2 17 49408
wc              2 18 35232
zombie          2 19 32536
console         3 22 0
$
```

- Press "**Ctrl+p**", to make kernel to print information about each process

```
$
1 init
2 sh

$
```

Press "enter" key to come out from process state list.

- Press "**Ctrl + a**" followed by "**x**", to instruct kernel to print information about each process

```
$
$ QEMU: Terminated
:/mnt/d/ICT1012/xv6labs-w1$
```

Part 3: Using gdb (GNU Debugger)

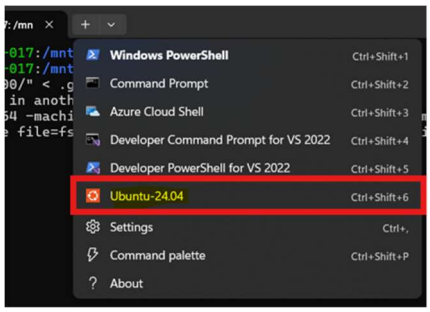
Starting "make qemu-gdb' from Ubuntu WSL terminal-1

Make sure you are in folder "*xv6labs-w1*" and execute "**make qemu-gdb**"

```
:/mnt/d/ICT1012/xv6labs-w1$ make qemu-gdb
sed "s/:1234/:26000/" < .gdbinit.tmpl-riscv > .gdbinit
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
nographic -global virtio-mmio.force-legacy=false -drive
file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-
bus.0 -S -gdb tcp::26000
```

Open another Ubuntu (WSL) terminal (terminal-2)

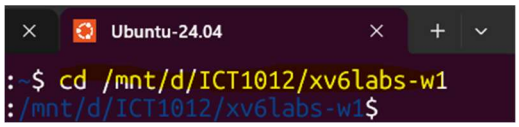
From Ubuntu terminal, click the **down arrow (v)** next to the "+" tab button. Select "**Ubuntu-24.04**" from the list of options



This will open a new tab running Ubuntu (WSL) terminal (terminal-2).

Navigate to "*xv6labs-w1*" directory by executing "**cd /mnt/d/ICT1012/xv6labs-w1**" in terminal-2.

```
:~$ cd /mnt/d/ICT1012/xv6labs-w1
:/mnt/d/ICT1012/xv6labs-w1$
```

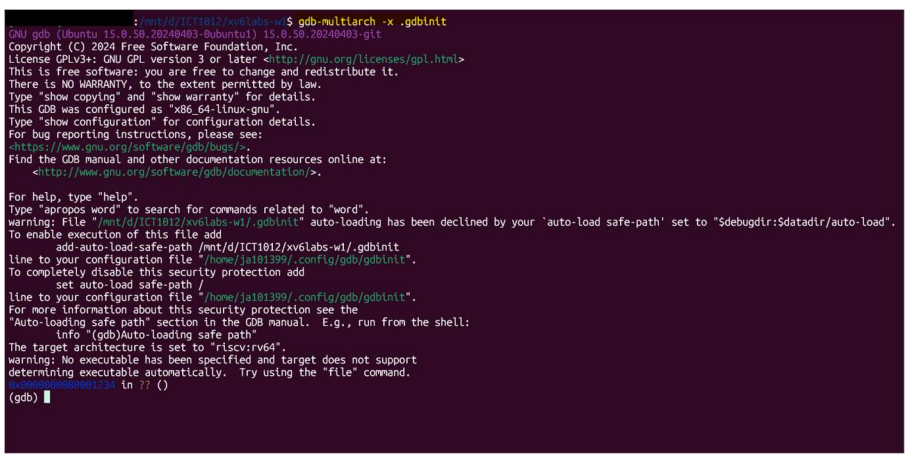


Starting 'gdb-multiarch (or riscv64-linux-gnu-gdb) -x .gdbinit' from WSL terminal-2

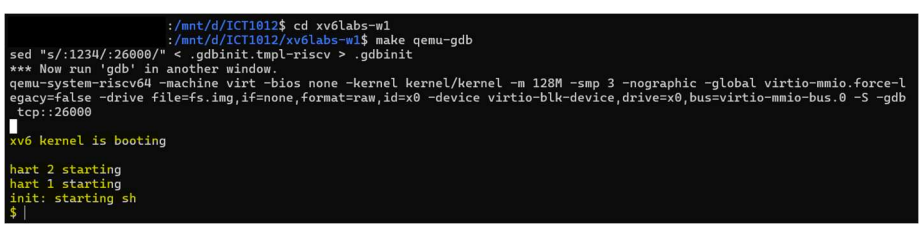
Connect GDB to QEMU by executing "**gdb-multiarch -x .gdbinit**" in terminal-2.

Note: This ".gdbinit" usually contains the target remote localhost:26000 symbol-file kernel/kernel. So, it auto-connects.

```
:/mnt/d/ICT1012/xv6labs-w1$ gdb-multiarch -x .gdbinit
```



At this point Ubuntu terminal-1 would have started "*qemu*":



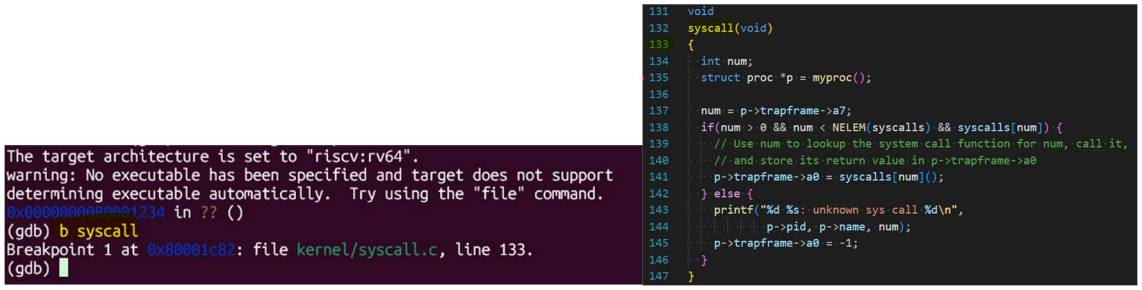
Getting familiar with “gdb”

Once the two terminals are open, type in terminal-2:

Execute “**b syscall**” to run up to the first break point in “*syscall*”

```
(gdb) b syscall
Breakpoint 1 at 0x80001c82: file kernel/syscall.c, line 133.
(gdb)
```

Screenshot:

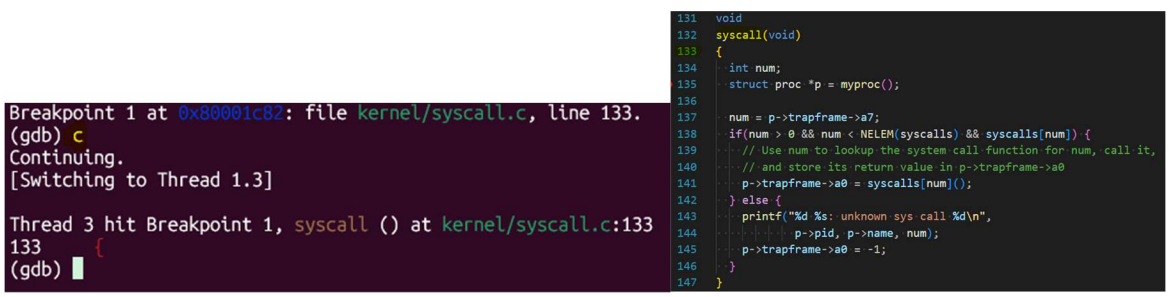


Execute “**c**” to continue

```
Breakpoint 1 at 0x80001c82: file kernel/syscall.c, line 133.
(gdb) c
Continuing.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:133
133     {
(gdb)
```

Screenshot:



Execute “**layout src**” to split the window in two, showing where “gdb” is in the source code.

```
Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:133
133     {
(gdb) layout src
```

Screenshot:

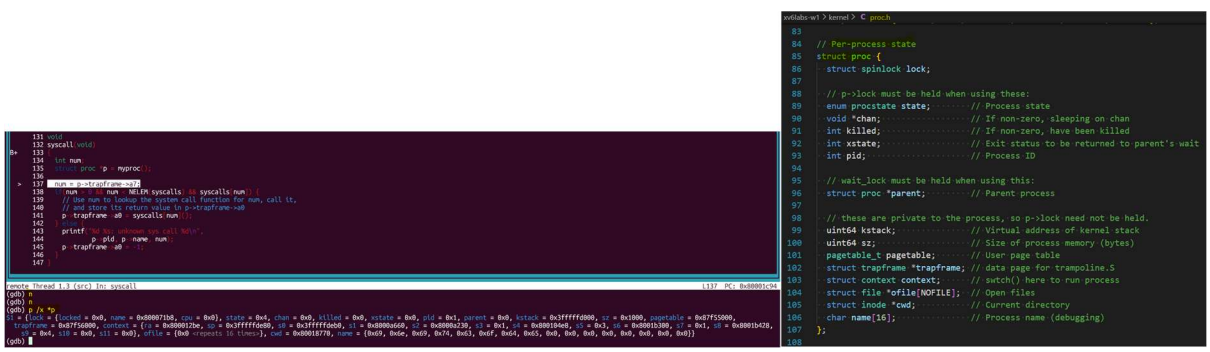
```
kernel/syscall.c
119 [SYS_sbrk]    sys_sbrk,
120 [SYS_sleep]   sys_sleep,
121 [SYS_uptime]  sys_uptime,
122 [SYS_open]    sys_open,
123 [SYS_write]   sys_write,
124 [SYS_mknod]   sys_mknod,
125 [SYS_unlink]  sys_unlink,
126 [SYS_link]    sys_link,
127 [SYS_mkdir]   sys_mkdir,
128 [SYS_close]   sys_close,
129 };
130
131 void
132 syscall(void)
B+> 133 {
134   int num;
135   struct proc *p = myproc();
136
137   num = p->trapframe->a7;
138   if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
139     // Use num to lookup the system call function for num, call it,
140     // and store its return value in p->trapframe->a0
141     p->trapframe->a0 = syscalls[num]();
142   } else {
143     printf("%d %s: unknown sys call %d\n",
144             p->pid, p->name, num);
145     p->trapframe->a0 = -1;
146   }
147 }
remote Thread 1.3 (src) In: syscall
(gdb)
```

Execute "**backtrace**" to print a stack backtrace

```
remote Thread 1.1 (src) In: syscall
(gdb) backtrace
#0  syscall () at kernel/syscall.c:133
#1  0x0000000080001a3e in usertrap () at kernel/trap.c:67
#2  0x0505050505050505 in ?? ()
(gdb)
```

Type "**n**" a few times to step past struct proc *p = myproc().

Once past this statement, type "**p /x *p**", which prints the current process's proc struct (see kernel/proc.h>) in hex.



The processor is running in supervisor mode, and we can print privileged registers such as "*sstatus*" (see RISC-V privileged instructions for a description):

```
(gdb) p /x $sstatus
$2 = 0x200000022
(gdb)
```

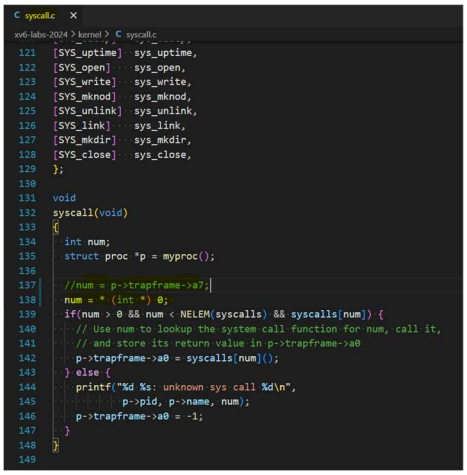
Quit out of "qemu":

- In terminal-2 window, press "**Ctrl + z**" or "**Ctrl + d**".
- In terminal-1 window, press "**Ctrl + a**" followed by "**x**".

Checking kernel code consistency (modifications) using "*gdb*"

1. The xv6 kernel code contains consistency checks whose failure causes the kernel to panic. You may find that your kernel modifications cause panic.
2. For example, at the beginning of "*syscall*" function in "*kernal/syscall.c*" replace the "num = p->trapframe->a7;" with "num = * (int *) 0;"

```
//num = p->trapframe->a7;
num = * (int *) 0;
```

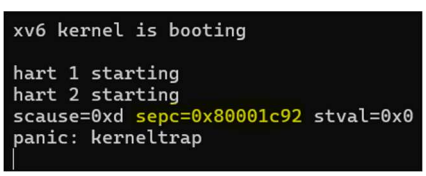


3. Build "*qemu*"

```
:/mnt/d/ICT1012/xv6labs-w1$ make qemu

...

xv6 kernel is booting
hart 2 starting
hart 1 starting
scause=0xd sepc=0x80001c92 stval=0x0
panic: kerneltrap
```



4. To track down the source of a kernel page-fault panic, search for the "*sepc*" (Supervisor Exception Program Counter) value printed for the panic you just saw in the file "*kernel/kernel.asm*", which contains the assembly for the compiled kernel.
5. In the above screenshot, the value of "sepc' is 0x80001c92. Your value may differ.
6. Stopping "qemu" or "qemu-gdb" in WSL terminal-1 from terminal-2.
   - Pressing "**Ctrl + a**" followed by "**x**" in Ubuntu terminal-1 will not be able to get out of QEMU since the terminal is fully occupied by QEMU's process and will not respond to the "interrupt" signal.
   - Execute "killall qemu-system-riscv64" from Ubuntu terminal-2. "killall" sends a termination signal to the QEMU process in terminal-1 from terminal-2.

     ```
     :/mnt/c/ICT1012/xv6labs-w1$ killall qemu-system-riscv64
     :/mnt/c/ICT1012/xv6labs-w1$
     ```

   - In terminal-1, check that QEMU has stopped.
     Screenshot:

```
:/mnt/c/ICT1012/xv6labs-w1$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-l
egacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
scause=0xd sepc=0x80001c92 stval=0x0
panic: kerneltrap
qemu-system-riscv64: terminating on signal 15 from pid 564 ()
:/mnt/c/ICT1012/xv6labs-w1$
```

7. Inspect the state of the processor and the kernel at the faulting instruction, fire up "gdb".
8. Tell GDB to load the symbols from the compiled kernel file by executing "**file kernel/kernel**"

```
(gdb) file kernel/kernel
Reading symbols from kernel/kernel...
```

```
The target architecture is set to "riscv:rv64".
warning: No executable has been specified and target does not support
determining executable automatically.  Try using the "file" command.
0x0000000000001000 in ?? ()
(gdb) file kernel/kernel
Reading symbols from kernel/kernel...
```

9. Set a breakpoint at the faulting "epc (exeception program counter)". From the earlier screenshot, "sepc" value is 0x80001c92. Execute "**b *0x80001c92**"

```
(gdb) b *0x80001c92
Breakpoint 1 at 0x80001c92: file kernel/syscall.c, line 138.
```

```
(gdb) b *0x80001c92
Breakpoint 1 at 0x80001c92: file kernel/syscall.c, line 138.
(gdb)
```

10. Execute "**layout asm**" to split the window in two, showing where gdb is in the assembly instructions.

```
(gdb) layout asm
Continuing.
```

11. Execute "**c**" to continue

```
(gdb) c
Continuing.
[Switching to Thread 1.3]
Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:138
```

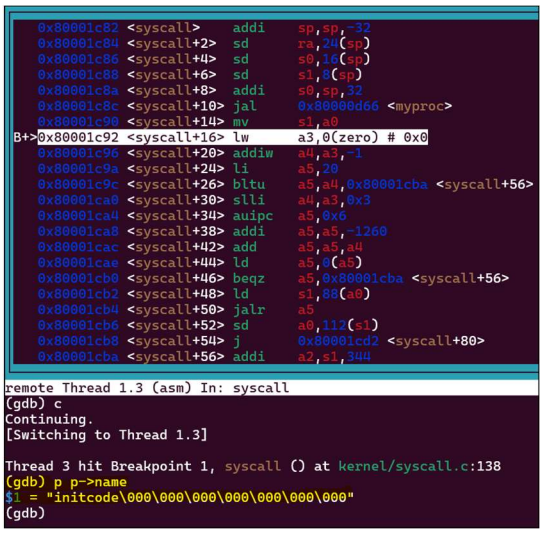
```
   0x80001c82 <syscall>     addi    sp,sp,-32
   0x80001c84 <syscall+2>   sd      ra,24(sp)
   0x80001c86 <syscall+4>   sd      s0,16(sp)
   0x80001c88 <syscall+6>   sd      s1,8(sp)
   0x80001c8a <syscall+8>   addi    s0,sp,32
   0x80001c8c <syscall+10>  jal     0x80000d66 <myproc>
   0x80001c90 <syscall+14>  mv      s1,a0
B+>0x80001c92 <syscall+16>  lw      a3,0(zero) # 0x0
   0x80001c96 <syscall+20>  addiw   a4,a3,-1
   0x80001c9a <syscall+24>  li      a5,20
   0x80001c9c <syscall+26>  bltu    a5,a4,0x80001cba <syscall+56>
   0x80001ca0 <syscall+30>  slli    a4,a3,0x3
   0x80001ca4 <syscall+34>  auipc   a5,0x6
   0x80001ca8 <syscall+38>  addi    a5,a5,-1260
   0x80001cac <syscall+42>  add     a5,a5,a4
remote Thread 1.3 (asm) In: syscall
(gdb) c
Continuing.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:138
(gdb)
```

12. Confirm that the faulting assembly instruction is the same as the one you found above.

13. The value 0xd (decimal 13) in the “scause” (Supervisor Cause) register indicates a Load Page Fault.
14. When the kernel panics with this code, it means the CPU attempted to read (load) data from a virtual address that is either not mapped in the page table or the permissions do not allow a load operation at that address.
15. To find out which user process was running when the kernel panicked, print the process's name by executing “p p->name”.

```
(gdb) p p->name
```



16. Quit “*gdb*” in terminal-2 by “**Ctrl + d**”
17. Execute “**killall qemu-system-riscv64”** from Ubuntu terminal-2 to send a termination signal to the QEMU process in terminal-1.

```
:/mnt/c/ICT1012/xv6labs-w1$ killall qemu-system-riscv64
:/mnt/c/ICT1012/xv6labs-w1$
```

18. Important: Set the “*kernel/syscall.c*” back to its original state.

```
num = p->trapframe->a7;
//num = * (int *) 0;
```